

Desarrollo Guiado por pruebas TDD

PRÁCTICA 5 MAP:

Test de Unidad con NUnit

Desarrollo guiado por pruebas (TDD)

- Se empiezan programando la batería de tests que probarán el código que queremos desarrollar.
- Una vez diseñados los tests, se desarrolla la aplicación.

Introducción a los tests de unidad

- Tests de unidad:
 - Son la base del TDD.
- Existen varios frameworks que facilitan la implementación de tests de unidad en nuestro código.
- Y también entornos de desarrollo que los integran para poder usarlos con mayor comodidad.
 - Vamos a practicar con uno de estos frameworks: NUnit.
 - <https://nunit.org/>
 - Y seguiremos usando nuestro entorno habitual de desarrollo (Visual Studio), añadiendo los paquetes necesarios para soportar Nunit.

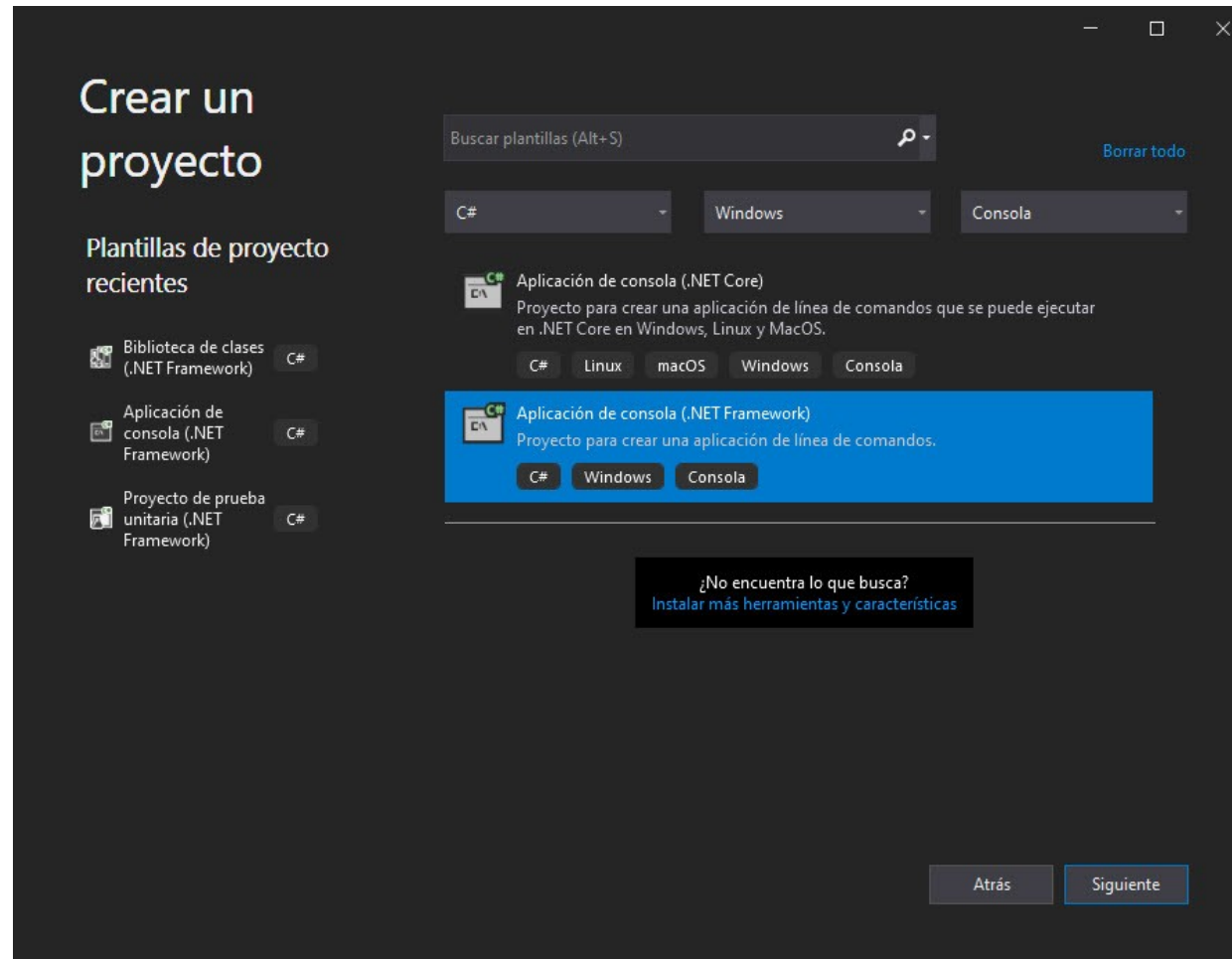
Tests de Unidad para la clase Lista

- Vais a partir de la solución PracticaBase que se os ha facilitado.
 - En ella se implementa una clase Lista, dentro del namespace Listas.
 - Los primeros tests de unidad para practicas con el framework los crearemos para esta clase.
- También podéis crear un nuevo proyecto de consola y copiar y pegar el código que se os facilita en Lista.cs

Crea un nuevo proyecto de consola

- Es necesario que sea un proyecto para framework .NET
 - **Aplicación de consola (.NET Framework)**

Aplicación de consola (.NET Framework)



Configure su nuevo proyecto

Aplicación de consola (.NET Framework) C# Windows Consola

Nombre del proyecto

P5

Ubicación

D:\Docencia\Curso 2021-22\MAP\TU\

Nombre de la solución ⓘ

P5

☒ Colocar la solución y el proyecto en el mismo directorio

Framework

.NET Framework 4.7.2

Atrás

Crear

Añadimos el fichero Lista.cs a ProyectoBase

```
using System;

namespace Listas {
    /// <summary>
    /// Lista para guardar números enteros.
    /// Se han de implementar todos los métodos que aquí aparecen
    /// </summary>
    public class Lista {

        /// <summary>
        /// Constructora de la lista.
        /// Construye una lista sin elementos
        /// </summary>
        public Lista() {
        }

        /// <summary>
        /// Número de elementos de la lista
        /// </summary>
        /// <returns>Devuelve el número de elementos (0 si es vacía)
        </returns>
        public int NumElems() {
        }
    }
}
```


Proyecto para las pruebas

- Incorporamos a nuestra solución un nuevo proyecto, que es el que incluirá los tests para probar la clase Lista.
 1. Pulsa con el botón derecho sobre el nombre de la solución, en el **Explorador de Soluciones**.
 2. Elige la opción **Agregar – Nuevo Proyecto**.
 3. Selecciona el tipo de proyecto **Biblioteca de clases (.NET Framework)**.

La versión del framework .NET debería ser la misma que la del proyecto base (verifícalo en Propiedades).
 4. Llama a este nuevo proyecto **Tests**

		 Compilar solución Ctrl+Mayús.+B Recompilar solución Limpiar solución Análisis y limpieza de código Compilación por lotes... Administrador de configuración...
		 Administrar paquetes NuGet para la solución...  Restaurar paquetes de NuGet
		 Ejecutar pruebas Depurar pruebas
		 Nueva vista de Explorador de soluciones Dependencias del proyecto... Orden de compilación del proyecto...
Nuevo proyecto...		Agregar
Proyecto existente...		 Establecer proyectos de inicio...
Sitio web existente...		 Create Git Repository...
 Nuevo elemento... Ctrl+Mayús.+A		 Pegar Ctrl+V
 Elemento existente... Mayús.+Alt+A		 Cambiar nombre F2
 Nueva carpeta de soluciones		Copiar ruta de acceso completa
Archivo de configuración de la instalación		 Abrir carpeta en el Explorador de archivos Guardar como filtro de soluciones Ocultar proyectos sin cargar
 Nuevo EditorConfig		 Propiedades Alt+Entrar

Agregar un nuevo proyecto

Plantillas de proyecto recientes

 Aplicación de consola (.NET Framework) C#

 Biblioteca de clases (.NET Framework) C#

 Proyecto de prueba unitaria (.NET Framework) C#

biblio



Borrar todo

C#

Windows

Consola

No se ha encontrado ninguna coincidencia exacta.

Otros resultados basados en su búsqueda



Biblioteca de clases (.NET Standard)

Proyecto para crear una biblioteca de clases para .NET Standard.

C#

Android

iOS

Linux

macOS

Windows

Biblioteca



Biblioteca de clases (.NET Standard)

Proyecto para crear una biblioteca de clases para .NET Standard.

Visual Basic

Android

iOS

Linux

macOS

Windows

Biblioteca



Biblioteca de clases (.NET Framework)

Proyecto para crear una biblioteca de clases de C# (.dll).

C#

Windows

Biblioteca



Biblioteca de clases (.NET Core)

Proyecto para crear una biblioteca de clases para .NET Core.

C#

Linux

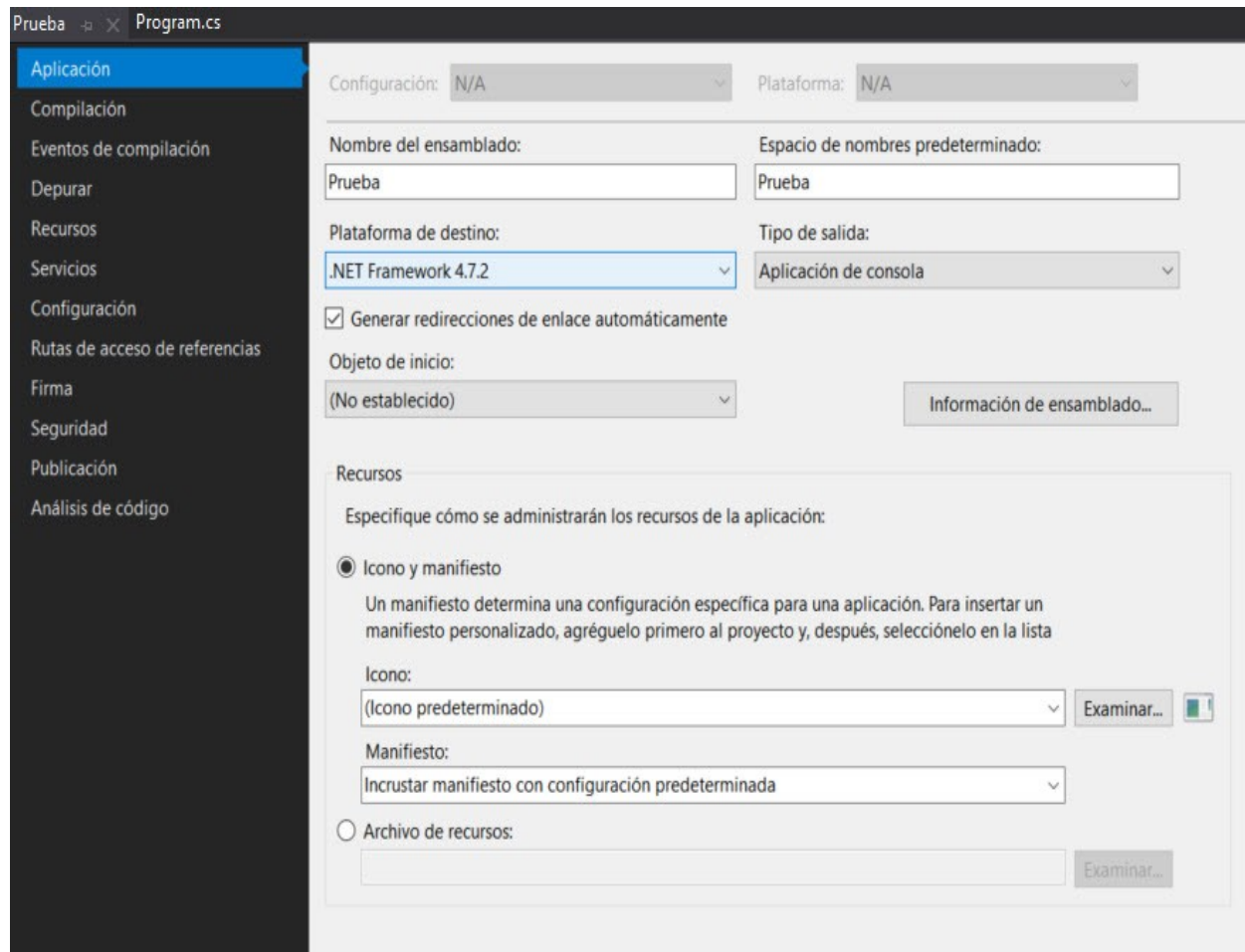
macOS

Windows

Biblioteca

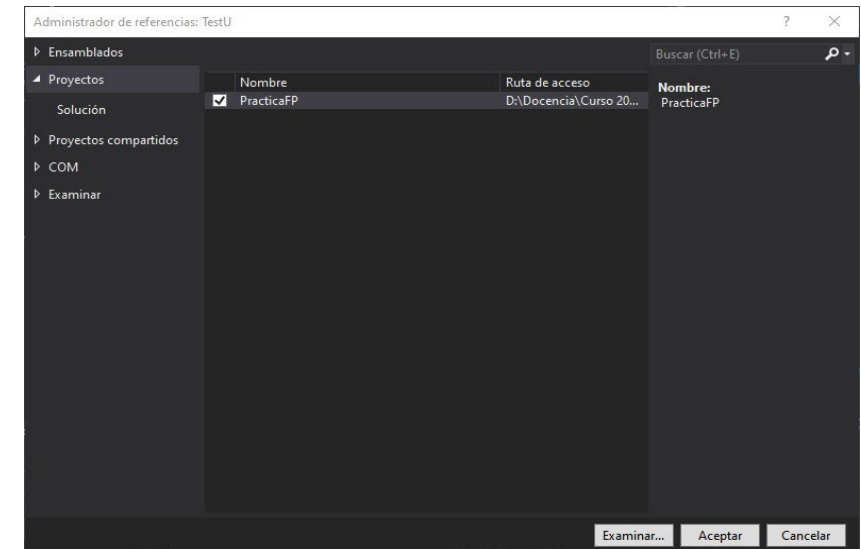
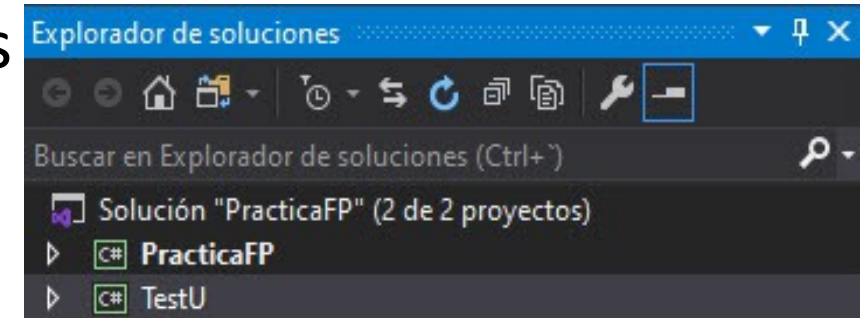
Siguiente

Propiedades: versión de .NET Framework



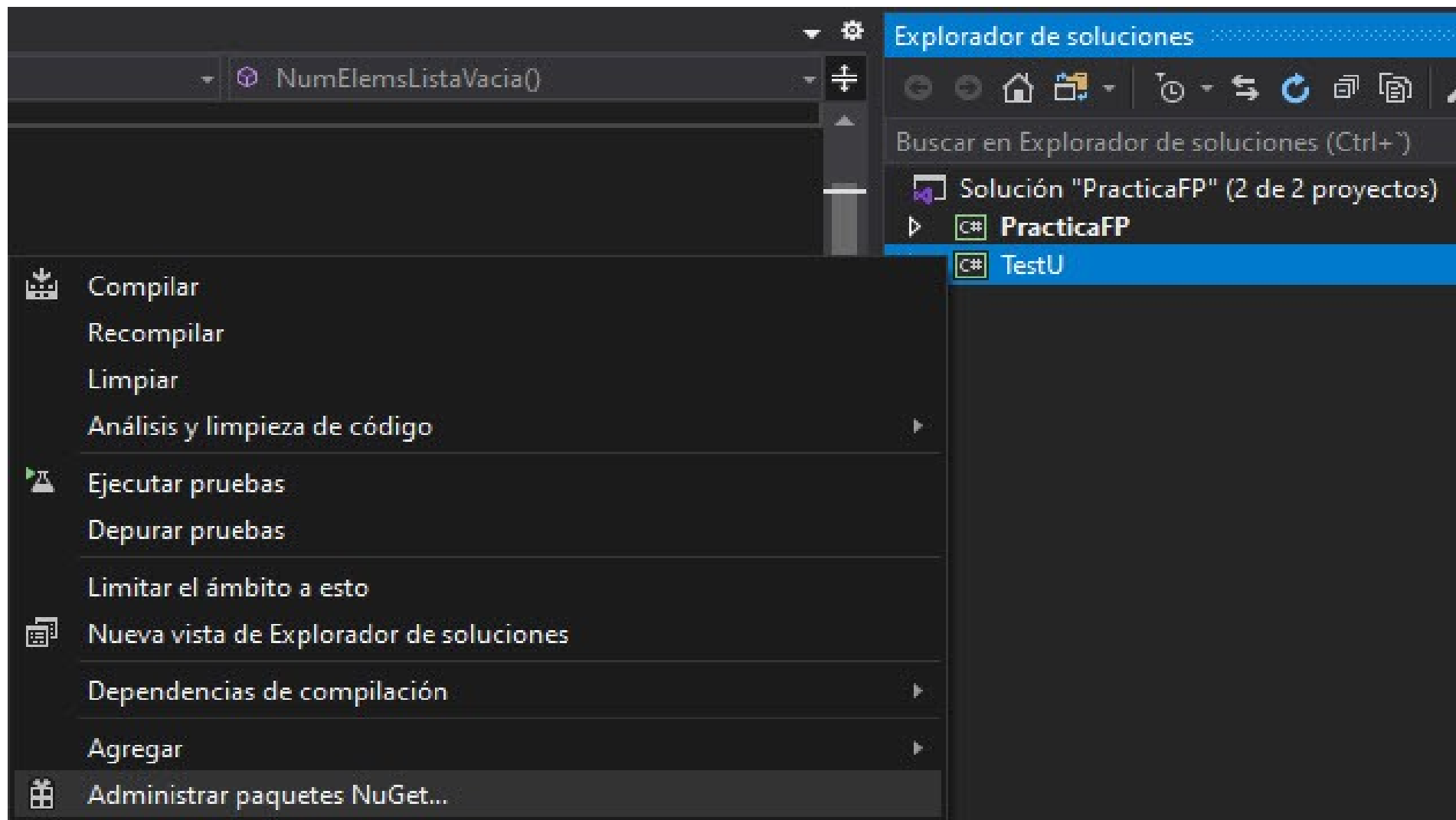
Dependencias entre proyectos

- Ahora en la solución aparecerán dos proyectos
 - Cada uno de ellos se compila de manera independiente, porque a priori no existe relación entre ellos.
 - Si queremos que este nuevo proyecto de pruebas use el código del proyecto que queremos probar, hay que añadir una dependencia.
- 1. Pulsa con el botón derecho encima del nombre del proyecto **Test** y elige la opción **Agregar-Referencia**,
- 2. En la sección **Proyectos**, marca el proyecto base, que contiene el código fuente que queremos probar.



Instalación de NUnit

- Ahora instalaremos los paquetes del framework Nunit que vamos a usar para implementar los tests de unidad
 1. Pulsamos con el botón derecho sobre el nombre del proyecto Tests y seleccionamos **Administrar paquetes NuGet**
 2. En la ventana que se abre buscamos en el cuadro de búsqueda de la pestaña **Examinar** los siguientes paquetes:
 - **NUnit3TestAdapter**
 - **Nunit**
 3. Seleccionamos cada uno de ellos y los instalamos en Visual Studio.



NuGet: TestU* ✕ Class1.cs Program.cs

Examinar Instalado Actualizaciones

nunit ✕ 🔄 ☐ Incluir versión preliminar

Administrador de paquetes NuGet: TestU

Origen del paquete: nuget.org ⚙️

 NUnit 🔍 por Charlie Poole, Rob Prouse, 164M descargas NUnit is a unit-testing framework for all .NET languages with a strong TDD focus.	v3.13.2
 NUnit3TestAdapter 🔍 por Charlie Poole, Terje Sandstrom, 106M descargas NUnit3 adapter for running tests in Visual Studio and DotNet. Works with NUnit 3.x, use the NUnit 2 adapter for 2.x tests.	v4.2.1

 **NUnit** 🔍 🌐 nuget.org

Versión: Versión estable más reciente 3.13.2 ⌵ Instalar

⌵ Opciones

NuGet: TestU* ✕ Class1.cs Program.cs

Examinar Instalado Actualizaciones

nunit ✕ 🔄 ☐ Incluir versión preliminar

 NUnit 🔍 por Charlie Poole, Rob Prouse, 164M descargas NUnit is a unit-testing framework for all .NET languages with a strong TDD focus.	v3.13.2
 NUnit3TestAdapter 🔍 por Charlie Poole, Terje Sandstrom, 106M descargas NUnit3 adapter for running tests in Visual Studio and DotNet. Works with NUnit 3.x, use the NUnit 2 adapter for 2.x tests.	v4.2.1
 NUnit.ConsoleRunner 🔍 por Charlie Poole, Rob Prouse, 28,2M descargas Console runner for the NUnit 3 unit-testing framework, without any extensions.	v3.15.0
 NUnit.Console 🔍 por Charlie Poole, Rob Prouse, 15,2M descargas Console runner for the NUnit 3 unit-testing framework with selected extensions.	v3.15.0

Paquetes instalados

NuGet: TestU Class1.cs Program.cs

Examinar Instalado Actualizaciones

nunit x ↕  ☐ Incluir versión preliminar

 **NUnit** por Charlie Poole, Rob Prouse v3.13.2
NUnit is a unit-testing framework for all .NET languages with a strong TDD focus.

 **NUnit3TestAdapter** por Charlie Poole, Terje Sandstrom v4.2.1
NUnit3 adapter for running tests in Visual Studio and DotNet. Works with NUnit 3.x, use the NUnit 2 adapter for 2.x tests.

Ejemplo de código de un test

1. Es el proyecto Test, usamos el archivo .cs que hay en la base y lo renombramos como **TestLista.cs**
2. Añade una directiva **using Listas;** al principio del archivo.

Si salta un error puedes revisar los siguiente:

- La clase Lista del proyecto base está dentro del **namespace Listas**.
 - Si no es así, añade al espacio de nombres la clase **Lista**
- Comprueba que has realizado los pasos correctamente a la hora de añadir una dependencia.

En caso de error relacionado con la clase

Lista

- Revisa que aparezca la palabra `public` delante de la definición de dicha clase:

`public class Lista`

- Revisa que tu clase tenga el método `NumElems`
 - O cámbialo por el nombre que hayas usado en tu código

Contenido de TestLista.cs

```
using System;
using NUnit.Framework;
using Listas;

namespace Test {
    [TestFixture]
    public class TestLista {
        [Test]
        public void NumElemsListaVacía() {
            // Arrange
            Lista unaLista = new Lista();
            // Act
            int numElemsLista = unaLista.NumElems();
            // Assert
            Assert.That( numElemsLista,
                          Is.EqualTo(0),
                          "ERROR: NumElems no devuelve 0 con lista vacía");
        }
    }
}
```

Explicación del código de los test

- Una clase que contiene una batería de test que prueban los métodos de otra clase base, ha de tener el atributo **[TestFixture]**.
- Cada método de esta clase que representa un test de unidad, ha de tener el atributo **[Test]**.
 - Si no aparece este atributo delante del método, no será ejecutado por la librería de tests de unidad.
- Cada una de las instrucciones que comienzan con **Assert** son los asertos:
 - Sirven para especificar la condición que se ha de cumplir en la prueba
 - Si no se cumple, se lanza un error y se muestra el mensaje que aparece como último parámetro.

Significado del ejemplo

- **Assert.That**

- Sintaxis de los asertos en el modelo basado en restricciones que vamos a usar.
- Lo usaremos con 3 parámetros:
 1. El valor que vamos a comprobar (i.e. el resultado de la ejecución del método que estamos probando): **numElemsLista**
 2. La restricción que se comprueba. En nuestro caso **Is.EqualTo(0)** comprueba que el 1er parámetro es 0.
 3. Si el test falla (**numElemsLista != 0**), se devuelve el mensaje que figura como tercer parámetro. Es importante que sea significativo, para identificar claramente qué ha fallado. Por ejemplo:

ERROR: numElemsLista no devuelve 0 con lista vacía.

Ejecución de lo tests

1. Muestra la ventana del explorador de pruebas seleccionándolo en el menú

Prueba >> Explorador de pruebas

1. En esa ventana puedes pulsar el botón con dos flechas verdes más a la izquierda para ejecutar todas las pruebas.
2. Si todo ha ido bien, el Explorador de pruebas nos muestra un tic verde en las pruebas correctas.

Explorador de pruebas para los TestLista

▶▶

↺

✖

🧪 1

✅ 1

❌ 0

📄

⌵

📋

⚙️

▶

🔍

Buscar en el explorador de pruebas

Prueba	Duración	Rasgos	Mensaje de error
▶ ✅ TestU (1)	194 ms		
▶ ✅ TestU (1)	194 ms		
▶ ✅ TestLista (1)	194 ms		
▶ ✅ NumElemListaVacia	194 ms		

Resumen del grupo

TestU

Pruebas en grupo: 1

⌚ Duración total: 194 ms

Salidas

✅

1 Correcta

Explorador de pruebas

PowerShell para desarrolladores

Lista de errores

Salida

¿Qué hay que hacer en la P5?

- Todos los detalles en el enunciado de la práctica.